

베이지안 기계학습

Variational Autoencoder

조성일

인하대학교
통계학과

Table of contents

1. Introduction
2. Principal components analysis
3. Autoencoders
4. Variational autoencoders
5. Example

Introduction

Motivation

- ▶ A common form of unsupervised learning is **dimensionality reduction**, in which we learn a mapping from the high-dimensional space to a low-dimensional space.
- ▶ The goals of dimensionality reduction are two-fold:
 - ▶ data visualization
 - ▶ preprocessing step for other kinds of learning algorithms

Principal components analysis

PCA: definition I

- ▶ Principal components analysis (PCA) is the simplest and most widely used form of dimensionality reduction.
- ▶ The basic idea is to find a **linear** and **orthogonal** projection of the high dimensional data $x \in \mathbb{R}^D$ to a low dimensional subspace $z \in \mathbb{R}^L$, where $L \ll D$.

PCA: definition II

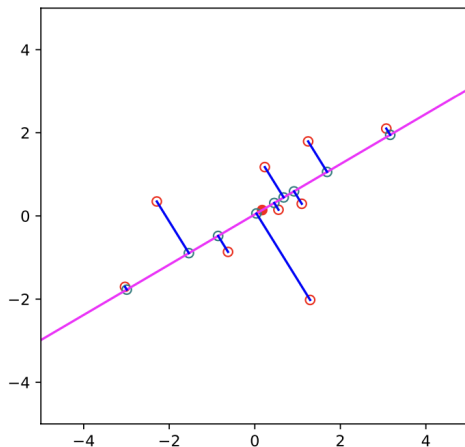


Figure: Murphy (2022). Figure 20.1 in Chapter 20.

PCA: algorithm I

- ▶ Let $\mathcal{D} = \{x_n : n = 1, \dots, N\}$ be an (unlabeled) dataset, where $x_n \in \mathbb{R}^D$ and $\bar{x} = \frac{1}{N} \sum_{n=1}^N x_n = 0$.
- ▶ PCA finds an orthogonal matrix W to approximate each x_n using a low dimensional representation, $z_n \in \mathbb{R}^L$ by minimizing the reconstruction error given as

$$\mathcal{L}(W, Z) = \frac{1}{N} \sum_{n=1}^N \|x_n - Wz_n\|^2.$$

PCA: algorithm II

- The optimal solution is obtained by setting $\hat{W} = U_L$, where U_L contains the L eigenvectors with largest eigenvalues of the empirical covariance matrix

$$\hat{\Sigma} = \frac{1}{N} X^\top X,$$

where $X = (x_1^\top, \dots, x_N^\top)^\top \in \mathbb{R}^{N \times D}$.

- **Note.** *Minimizing* the reconstruction error is equivalent to *maximizing* the variance of the projected data.

PCA: algorithm III

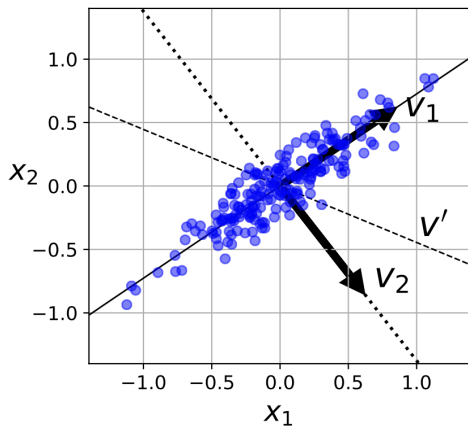


Figure: Murphy (2022). Figure 20.4 in Chapter 20.

PCA: Example

► Olivetti face database

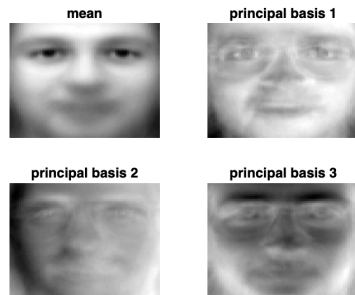


Figure: Murphy (2022). Figure 20.3 in Chapter 20.

Autoencoders

AE: definition I

- ▶ PCA can be thought of as learning a linear mapping from $x \rightarrow z$, called the **encoder**, (f_e), and learning another linear mapping $z \rightarrow x$, called the **decoder**, (f_d).
- ▶ Reconstruction error:

$$\mathcal{L}(W) \triangleq \frac{1}{N} \sum_{n=1}^N \|x_n - \text{decode}(\text{encode}(x_n; W); W)\|_2^2.$$

AE: definition II

- ▶ An autoencoder (AE) considers nonlinear mappings for both the **encoder** and the **decoder**.
- ▶ Particularly, AE implements these nonlinear mappings by using neural networks.

AE: definition III

- Denote the overall reconstruction function as

$$r(x) = f_d(f_e(x)),$$

where $f_d(\cdot)$ and $f_e(\cdot)$ are neural networks.

- AE minimizes the reconstruction error

$$\mathcal{L}(\theta) = \|r(x) - x\|_2^2.$$

AE: definition IV

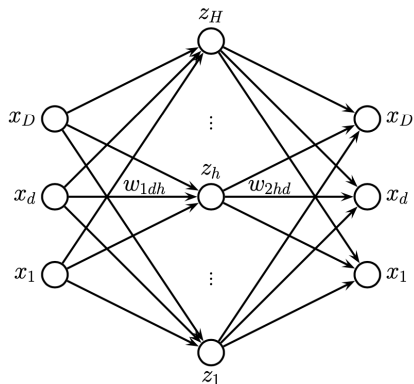


Figure: Murphy (2022). Figure 20.16 in Chapter 20.

AE: Bottleneck autoencoder I

- ▶ To prevent the degenerate solution (learning the identity function), we have to restrict the model in some way.
- ▶ The simplest approach is to use a narrow bottleneck layer, with $L \ll D$.
- ▶ This is called an undercomplete representation.

AE: Bottleneck autoencoder II

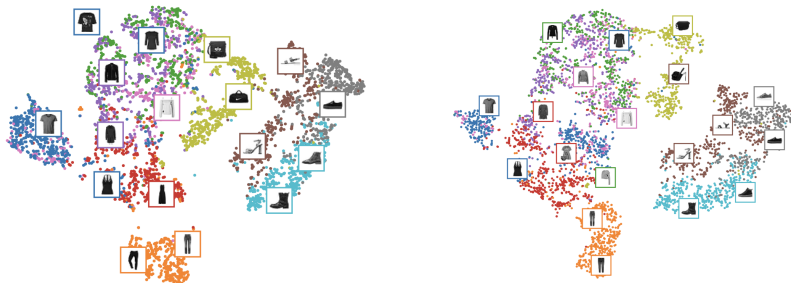


Figure: Murphy (2022). Figure 20.18 in Chapter 20.

AE: Regularization

- ▶ The other approach is to use $L \gg D$, known as an overcomplete representation, but to impose some kind of regularization.
 - ▶ adding noise to the inputs (denoising autoencoder)
 - ▶ forcing the activations of the hidden units to be sparse (sparse autoencoder)
 - ▶ imposing a penalty on the derivatives of the hidden units (contractive autoencoder)

AE: Denoising autoencoder I

- ▶ A denoising autoencoder (DAE) adds noises to the inputs, and then reconstructs a clean (uncorrupted) version of the original inputs.
- ▶ For example, we can train a DAE using Gaussian corruption and squared error reconstruction, i.e., we can use $p(\tilde{x} | x) = \mathcal{N}(\tilde{x} | x, \sigma^2 I)$ and $l(x, r(\tilde{x})) = \|r(\tilde{x}) - x\|_2^2$.

AE: Denoising autoencoder II

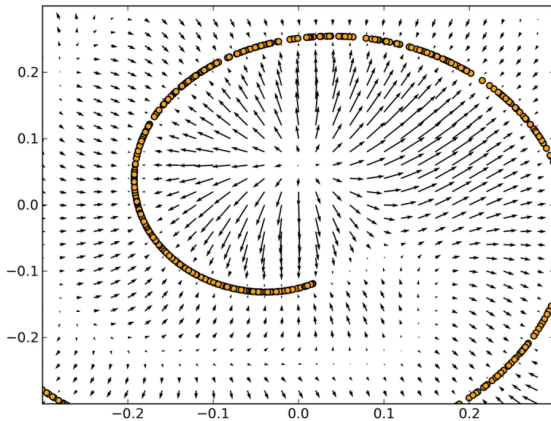


Figure: Murphy (2022). Figure 20.20 in Chapter 20.

AE: Denoising autoencoder III



Figure: Murphy (2022). Figure 20.20 in Chapter 20.

Variational autoencoders

VAE: Introduction I

- ▶ A variational autoencoder (VAE) can be thought of as a probabilistic version of a deterministic autoencoder.
- ▶ The principal advantage is that a VAE is a generative model that can create new samples, whereas an autoencoder just computes embeddings of input vectors.

VAE: Introduction II

- ▶ A VAE defines a generative model of the form

$$p_{\theta}(z, x) = p_{\theta}(z)p_{\theta}(x | z),$$

where p_{θ} is usually a Gaussian, and $p_{\theta}(x | z)$ is usually a product of exponential family distributions (e.g., Gaussians or Bernoullis) with parameters computed by a neural network decoder, $f_d(z)$.

VAE: Introduction III

- In addition, a VAE fits the following recognition model to perform approximate posterior inference

$$q_{\phi}(z \mid x) = q(z \mid f_e(x; \phi)) \approx p_{\theta}(z \mid x),$$

where $q_{\phi}(z \mid x)$ is usually a Gaussian, with parameters computed by a neural network encoder $f_e(x; \phi)$:

$$\begin{aligned} q_{\phi}(z \mid x) &= \mathcal{N}(z \mid \mu, \text{diag}(\exp(l))) \\ (\mu, l) &= f_e(x; \phi), l = \log \sigma. \end{aligned}$$

VAE: Introduction IV

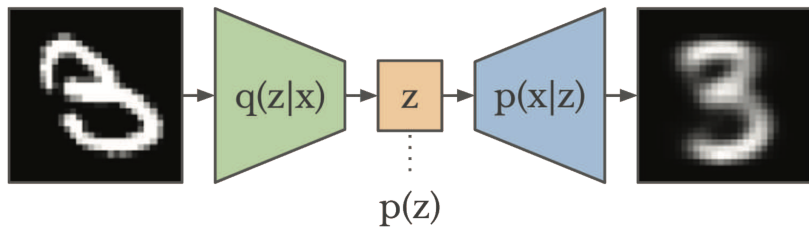


Figure: Murphy (2023). Figure 21.1 in Chapter 21.

VAE: Comparison I

- ▶ The main advantage of a VAE over a deterministic autoencoder is that it defines a proper generative model.
- ▶ A generative model can create sensible-looking novel images by decoding prior samples $z \sim \mathcal{N}(0, I)$.
- ▶ By contrast, an autoencoder only knows how to decode latent codes derived from the training set, so does poorly when fed random inputs.

VAE: Comparison II

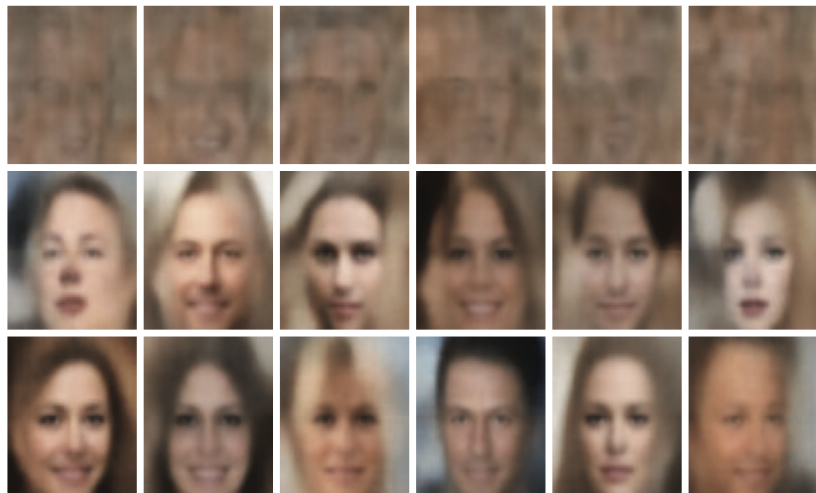


Figure: Murphy (2023). Figure 21.3 in Chapter 21.

Remarks: Generative model I

- ▶ A generative model is a joint probability distribution $p(x)$, for $x \in \mathcal{X}$.
- ▶ In some cases, the model may be conditioned on inputs or covariates $c \in \mathcal{C}$, which gives rise to a conditional generative model of the form $p(x \mid c)$.

Remarks: Generative model II

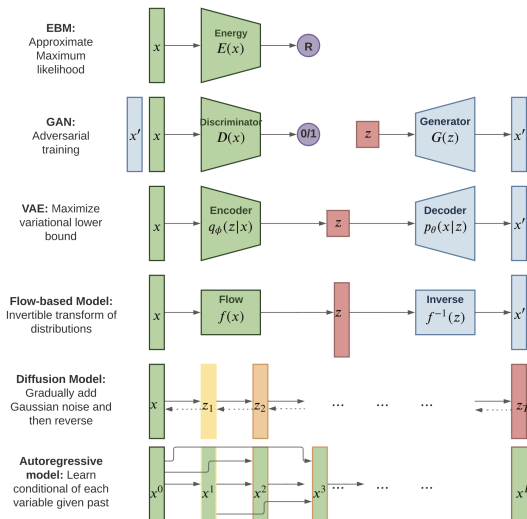


Figure: Murphy (2023). Figure 20.1 in Chapter 20.

Remarks: Generative model III

- ▶ There are several different kinds of tasks that we can use generative models for.
 - ▶ **generating data**: generating (create) new data samples (generative AI).
 - ▶ **density estimation**: evaluating the probability of an observed data vector, i.e., computing $p(x)$.
 - ▶ **imputation**: filling in missing values of a data vector or data matrix.
 - ▶ **latent space interpolation**: interpolating between existing datapoints in latent space.
 - ▶ ...

Example

Libraries

```
import os

import numpy as np
import torch
from pyro.contrib.examples.util import MNIST
import torch.nn as nn
import torchvision.transforms as transforms

import pyro
import pyro.distributions as dist
import pyro.contrib.examples.util # patches torchvision
from pyro.infer import SVI, Trace_ELBO
from pyro.optim import Adam
```

Example: Dataset I

```
def setup_data_loaders(batch_size=128):  
    root = './data'  
    trans = transforms.ToTensor()  
    train_set = MNIST(root=root, train=True, transform = trans,  
                      download=True)  
    test_set = MNIST(root=root, train=False, transform=trans)  
  
    kwargs = {'num_workers': 1}  
    train_loader = torch.utils.data.DataLoader(dataset=train_set,  
                                                batch_size=batch_size, shuffle=True, **kwargs)  
    test_loader = torch.utils.data.DataLoader(dataset=test_set,  
                                              batch_size=batch_size, shuffle=False, **kwargs)  
  
    return train_loader, test_loader
```

Example: Dataset II

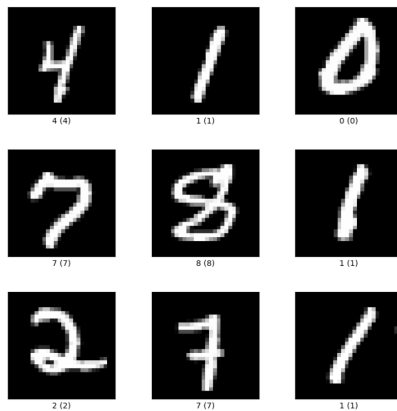


Figure: Modified National Institute of Standards and Technology (MNIST)

Example: Decoder

```
class Decoder(nn.Module):
    def __init__(self, z_dim, hidden_dim):
        super().__init__()

        # setup the two linear transformations used
        self.fc1 = nn.Linear(z_dim, hidden_dim)
        self.fc21 = nn.Linear(hidden_dim, 784)

        # setup the non-linearities
        self.softplus = nn.Softplus()
        self.sigmoid = nn.Sigmoid()

    def forward(self, z):
        # define the forward computation on the latent z
        # first compute the hidden units
        hidden = self.softplus(self.fc1(z))

        # return the parameter for the output Bernoulli
        # each is of size batch_size x 784
        loc_img = self.sigmoid(self.fc21(hidden))
        return loc_img
```

Example: Encoder

```
class Encoder(nn.Module):
    def __init__(self, z_dim, hidden_dim):
        super().__init__()
        # setup the three linear transformations used
        self.fc1 = nn.Linear(784, hidden_dim)
        self.fc21 = nn.Linear(hidden_dim, z_dim)
        self.fc22 = nn.Linear(hidden_dim, z_dim)
        # setup the non-linearities
        self.softplus = nn.Softplus()

    def forward(self, x):
        # define the forward computation on the image x
        # first shape the mini-batch to have pixels in the rightmost dim
        x = x.reshape(-1, 784)
        # then compute the hidden units
        hidden = self.softplus(self.fc1(x))
        # then return a mean vector and a square root covariance
        # each of size batch_size x z_dim
        z_loc = self.fc21(hidden)
        z_scale = torch.exp(self.fc22(hidden))
        return z_loc, z_scale
```

Example: Model

```
# define the model  $p(x|z)p(z)$ 
def model(self, x):
    # register PyTorch module 'decoder' with Pyro
    pyro.module("decoder", self.decoder)
    with pyro.plate("data", x.shape[0]):
        # setup hyperparameters for prior  $p(z)$ 
        z_loc = x.new_zeros(torch.Size((x.shape[0], self.z_dim)))
        z_scale = x.new_ones(torch.Size((x.shape[0], self.z_dim)))

        # sample from prior
        # (value will be sampled by guide when computing the ELBO)
        z = pyro.sample("latent", dist.Normal(z_loc, z_scale).to_event(1))

        # decode the latent code z
        loc_img = self.decoder(z)

        # score against actual images
        pyro.sample("obs", dist.Bernoulli(loc_img).to_event(1),
                    obs=x.reshape(-1, 784))
```

Example: Guide

```
# define the guide (i.e. variational distribution)  $q(z|x)$ 
def guide(self, x):
    # register PyTorch module 'encoder' with Pyro
    pyro.module("encoder", self.encoder)
    with pyro.plate("data", x.shape[0]):
        # use the encoder to get the parameters used to define  $q(z|x)$ 
        z_loc, z_scale = self.encoder(x)

    # sample the latent code z
    pyro.sample("latent", dist.Normal(z_loc, z_scale).to_event(1))
```

Example: Training

```
def train(svi, train_loader):  
    # initialize loss accumulator  
    epoch_loss = 0.  
  
    # do a training epoch over each mini-batch x returned  
    # by the data loader  
    for x, _ in train_loader:  
        # do ELBO gradient and accumulate loss  
        epoch_loss += svi.step(x)  
  
    # return epoch loss  
    normalizer_train = len(train_loader.dataset)  
    total_epoch_loss_train = epoch_loss / normalizer_train  
    return total_epoch_loss_train
```

Example: Test

```
def evaluate(svi, test_loader):  
    # initialize loss accumulator  
    test_loss = 0.  
  
    # compute the loss over the entire test set  
    for x, _ in test_loader:  
        # compute ELBO estimate and accumulate loss  
        test_loss += svi.evaluate_loss(x)  
  
    normalizer_test = len(test_loader.dataset)  
    total_epoch_loss_test = test_loss / normalizer_test  
    return total_epoch_loss_test
```

Example: Fitting I

```
NUM_EPOCHS = 100
TEST_FREQUENCY = 5

train_loader, test_loader = setup_data_loaders(batch_size=256)

# clear param store
pyro.clear_param_store()

# setup the VAE
vae = VAE()

# setup the optimizer
optimizer = Adam({"lr": 1.0e-3})

# setup the inference algorithm
svi = SVI(vae.model, vae.guide, optimizer, loss=Trace_ELBO())
```

Example: Fitting II

```
train_elbo = []
test_elbo = []

# training loop
for epoch in range(NUM_EPOCHS):
    total_epoch_loss_train = train(svi, train_loader)
    train_elbo.append(-total_epoch_loss_train)

    if epoch % TEST_FREQUENCY == 0:
        # report test diagnostics
        total_epoch_loss_test = evaluate(svi, test_loader)
        test_elbo.append(-total_epoch_loss_test)
```

Remark

Remark

- ▶ Autoencoders are deterministic dimensionality reduction techniques using nonlinear mappings for both the **encoder** and the **decoder**.
- ▶ Variational autoencoders can be thought of as a probabilistic version of deterministic autoencoders.
- ▶ The principal advantage is that a VAE is a generative model that can create new samples, whereas an autoencoder just computes embeddings of input vectors.

References

- ▶ Pyro tutorials
- ▶ Murphy (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press.
- ▶ Murphy (2023). *Probabilistic Machine Learning: Advanced Topics*. MIT Press.